

MoPhones | Case Study

Data Analyst – Product & Credit

Prepared by: Maurice Otieno Juma

Submission: Slide Deck (12 slides max) +
Jupyter Notebook

Date: 20/04/2026

Analyzing credit portfolio health, Credit Outcomes Vs Customer Experience, and data quality to provide recommendations, actionable insights for MoPhones' product and credit strategy.

Data Loading

Datasets included for analysis:

- Credit portfolio snapshots (Jan, Mar, Jun, Sep, Dec)
- Customer demographics (DOB, income fields, employment duration)
- NPS survey responses linked to customer IDs

Sheets loaded and merged:

- **Sales Details** – transactional records of customer purchases
- **Gender** – demographic segmentation for portfolio analysis
- **Date of Birth** – used to derive age and age bands
- **Income Level** – combined with employment duration to calculate average monthly income and income bands

Final merged dataset: All credit snapshots, customer demographics, sales records, and NPS survey responses were harmonized using LoanID as the unique key. This unified dataset enables consistent monitoring of portfolio health,

segmentation by age and income, and exploration of the relationship between credit outcomes and customer experience.

In [2]:

```
import pandas as pd
import os
import openpyxl
from functools import reduce

# Set working directory
path = (
    r"C:\Users\mauri\OneDrive\Desktop\Documents\My
Documents\Job Application Documents\Mophones"
)
os.chdir(path)

# Load each snapshot without adding SnapshotDate
jan = pd.read_excel("Credit Data - 01-01-2025.xlsx");
jan["DATE"] = pd.to_datetime("2025-01-01")
mar = pd.read_excel("Credit Data - 30-03-2025.xlsx");
mar["DATE"] = pd.to_datetime("2025-03-30")
jun = pd.read_excel("Credit Data - 30-06-2025.xlsx");
jun["DATE"] = pd.to_datetime("2025-06-30")
sep = pd.read_excel("Credit Data - 30-09-2025.xlsx");
sep["DATE"] = pd.to_datetime("2025-09-30")
dec = pd.read_excel("Credit Data - 30-12-2025.xlsx");
```

Credit data shape: (71456, 33)

NPS shape: (3532, 17)

NPS shape: (3532, 17)

Data Integration

Datasets integrated for analysis:

- **Credit Portfolio Snapshots** – point-in-time views of balances, arrears, and repayment status across five reporting dates (Jan, Mar, Jun, Sep, Dec)
- **Customer Demographics** – including date of birth, income fields, and employment duration, used to derive age bands and income categories
- **NPS Survey Responses** – customer satisfaction scores linked to LoanID, enabling exploration of the relationship between credit outcomes and customer experience

Final merged dataset: All credit snapshots, customer demographics, sales records, and NPS survey responses were harmonized using LoanID as the unique key. This unified dataset provides a single source of truth for portfolio monitoring, segmentation by age and income, and linking repayment behaviour with customer satisfaction.

Objective: *Establish a consolidated dataset that supports consistent reporting, enables deeper operational insights, and strengthens decision-making around credit risk management and customer experience improvement.*

In [3]:

```
# Merge safely
customers = sales_details.merge(gender, on="LOANID",
how="left") \
                                .merge(dob, on="LOANID",
how="left") \
                                .merge(income, on="LOANID",
how="left")

print("Customers shape:", customers.shape)
# -----
# Step 5: Merge everything
# -----
full_data = credit.merge(customers, on="LOANID",
how="left") \
                .merge(nps, on="LOANID", how="left")

print("Full dataset shape:", full_data.shape)
```

Customers shape: (20747, 27)

Full dataset shape: (71466, 75)

Data Preparation

Derived fields created to enable standardized analysis:

- **Age Band** – calculated from each customer's Date of Birth at the snapshot reporting date. Grouped into categories (18–25, 26–35, 36–45, 46–55, 55+) to allow segmentation of repayment behaviour and risk patterns by age.
- **Average Monthly Income** – derived by summing all income fields and dividing by employment duration. This normalizes income across customers with different work histories, ensuring fair comparison of repayment capacity.
- **Income Band Categories** – customers are grouped into defined ranges (Below 5,000; 5,000–9,999; 10,000–19,999; 20,000–29,999; 30,000–49,999; 50,000–99,999; 100,000–149,999; 150,000+). These bands provide a structured view of affordability and help

identify segments with higher credit risk or repayment resilience.

Objective: *Transform raw demographic and income data into standardized analytical fields that support portfolio monitoring, customer segmentation, and evidence-based decision-making for credit risk management.*

In [4]:

```

# -----
# Step 1: Convert date columns
# -----
# Convert 'date_of_birth' and 'DATE' into proper
# datetime objects.
# Using errors="coerce" ensures invalid formats are
# safely converted to NaT.
full_data["date_of_birth"] =
pd.to_datetime(full_data["date_of_birth"], format="%Y-
%m-%d", errors="coerce")
full_data["DATE"] = pd.to_datetime(full_data["DATE"],
format="%Y-%m-%d", errors="coerce")

# -----
# Step 2: Age calculation
# -----
# Compute customer age at snapshot date.
# NOTE: Original code subtracted DATE from itself
# (always zero).
# Correct calculation subtracts year of birth from
# snapshot year.
full_data["Age"] = full_data["DATE"].dt.year -

```

	LOANID	AgeBand	AvgMonthlyIncome
IncomeBand			
2	recPCzNo2d9qK1VB3	Unknown	66866.623333
50,000-99,999			
6	rec6dwN71Phff6ZIW	Unknown	154915.223333
150,000+			
11	recZVxmo8rPb9fMZn	Unknown	262233.750000
150,000+			
29	recgDv0z8ZFP0gop0	Unknown	259276.500000
150,000+			
37	recTocDHUrhpcHoN7	Unknown	441349.393333
150,000+			

Portfolio Health Metrics

Selected indicators for monitoring credit portfolio performance:

- **Arrears Rate** – percentage of accounts past due. This highlights early signs of repayment stress and helps identify segments requiring proactive collections or customer support.
- **Default Rate** – percentage of accounts that have fully defaulted. This is a critical measure of portfolio risk exposure and informs provisioning, write-off strategies, and credit policy adjustments.
- **Outstanding Balance** – total value of loans currently active in the portfolio. This reflects the institution's overall credit exposure and is essential for liquidity planning and capital adequacy monitoring.
- **Repayment Coverage** – ratio of payments received to balances due. This indicator measures repayment discipline and portfolio sustainability, providing insight into

customer affordability and repayment behaviour.

Objective: *Track these metrics consistently across reporting snapshots to identify emerging risks, monitor repayment trends, and support evidence-based decision-making for credit risk management and customer engagement strategies.*

In [5]:

```

# Step 1: Convert date columns
# Ensure 'date_of_birth' and 'DATE' are proper datetime
objects.
full_data["date_of_birth"] =
pd.to_datetime(full_data["date_of_birth"], format="%Y-
%m-%d", errors="coerce")
full_data["DATE"] = pd.to_datetime(full_data["DATE"],
format="%Y-%m-%d", errors="coerce")

# Step 2: Age calculation
# Compute age at snapshot date (corrected formula).
full_data["Age"] = full_data["DATE"].dt.year -
full_data["date_of_birth"].dt.year

# Step 3: Age bands
# Group ages into categories for segmentation.
def age_band(age):
    if pd.isna(age): return "Unknown"
    elif age < 26: return "18-25"
    elif age < 36: return "26-35"
    elif age < 46: return "36-45"
    elif age < 56: return "46-55"
    else: return ""

```

	LOANID	AgeBand	AvgMonthlyIncome
IncomeBand			
627	rec1WfRVvT8H7sDZO	18-25	12147.086667
10,000-19,999			
2239	recYPntN91PIBz0oa	26-35	550066.100833
150,000+			
4696	recuWHCtT6fUmuc3M	26-35	50892.300000
50,000-99,999			
5176	recxvya1MRb0cgnQz	26-35	34665.815000
30,000-49,999			
6998	recY10IF9TZsk6eZ0	36-45	73583.635000
50,000-99,999			

Portfolio Health Trends

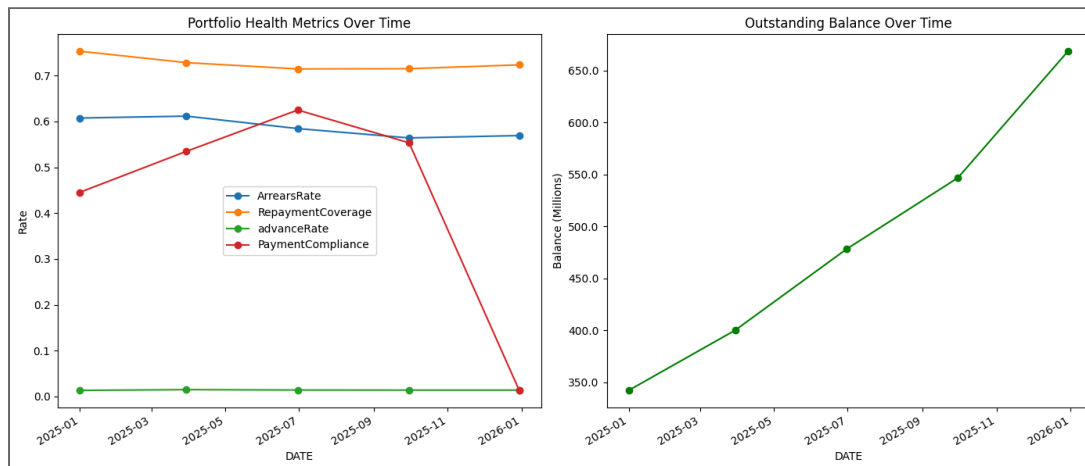
Key indicators of credit portfolio performance across reporting snapshots:

- **Arrears Rate** – proportion of accounts past due, signaling repayment stress and early risk exposure.
- **Repayment Coverage** – ratio of payments received to balances due, reflecting repayment discipline and sustainability.
- **Advance Rate** – share of balances covered by advance or prepayments, highlighting proactive customer behavior.
- **Payment Compliance** – actual vs expected payments, measuring adherence to repayment schedules.
- **Outstanding Balance** – total loan exposure, showing portfolio growth and liquidity requirements.

Objective: *Present a holistic view of portfolio health by combining repayment discipline, arrears, compliance, and exposure trends. This*

supports proactive risk management, forecasting, and evidence-based decision-making.

Illustration: *The charts below show how arrears, repayment coverage, advance rate, and payment compliance evolve over time, alongside the outstanding balance trend, providing a comprehensive picture of portfolio performance.*



In [6]:

```

import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# -----
# Step 1: Calculate portfolio health metrics
# -----
# - ArrearsRate: share of accounts past due
# - RepaymentCoverage: payments received ÷ balances due
# - OutstandingBalance: total loan exposure
# - AdvanceRate: proportion of balances
#   prepaid/advanced
# - PaymentCompliance: actual vs expected payments
# -----
arrears_rate = credit.groupby("DATE")
["ARREARS"].apply(lambda x: (x > 0).mean())
repayment_coverage = credit.groupby("DATE")
["TOTAL_PAID"].sum() / credit.groupby("DATE")
["TOTAL_DUE_TODAY"].sum()
outstanding_balance = credit.groupby("DATE")
["BALANCE"].sum()
advance_rate = (credit.groupby("DATE")["ADVANCE"].sum()
+ credit.groupby("DATE")["PREPAYMENT_AMOUNT"].sum()) /

```

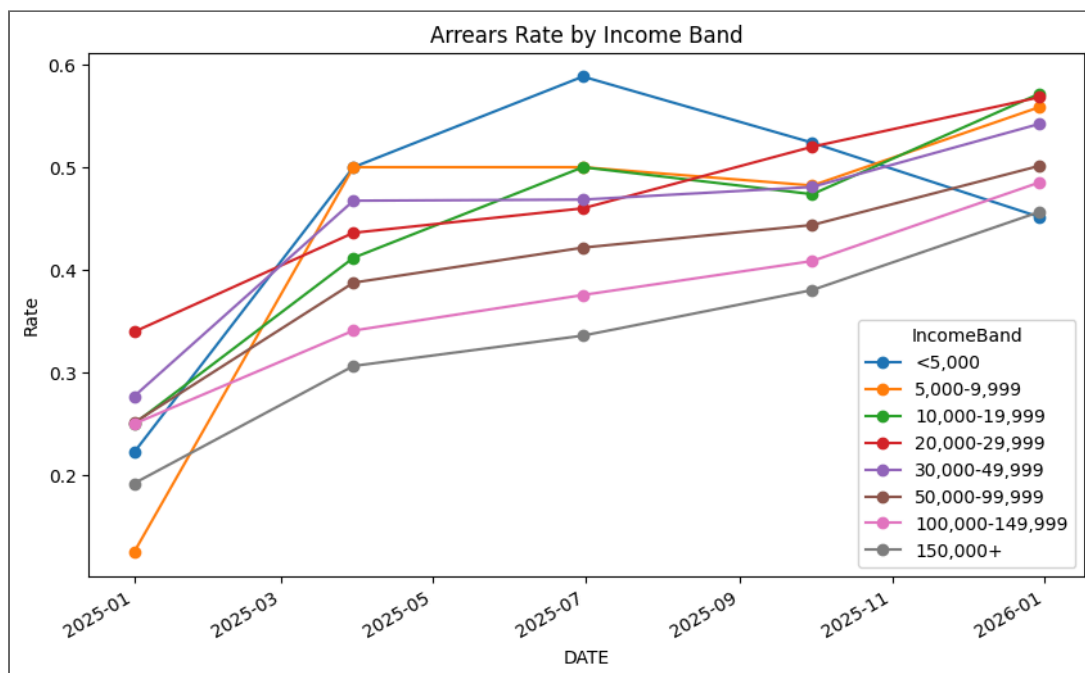
ArrearsRate	OutstandingBalance	Repa
ymmentCoverage	advanceRate	\
DATE		
2025-01-01	0.607499	342087292.0
0.753263	0.013286	
2025-03-30	0.611665	399987192.4
0.728353	0.014923	
2025-06-30	0.584479	478068036.4
0.714648	0.014065	
2025-09-30	0.564101	546661562.8
0.715136	0.013841	
2025-12-30	0.569232	668547758.2
0.723508	0.013835	

PaymentCompliance
DATE
2025-01-01
0.445495
2025-03-30
0.534812
2025-06-30
0.625038
2025-09-30
0.553470
2025-12-30
0.013367

Segment Analysis & Operational Implications

Segment Analysis:

Disaggregating arrears by **Income Band** reveals that lower-income customers consistently show higher arrears rates compared to the portfolio average. This divergence highlights concentrated repayment risk in vulnerable segments.



Operational Implications:

- Enable **targeted collections strategies** for high-risk segments, improving repayment outcomes.
- Provide **tailored repayment support** such as flexible plans or reminders to reduce

arrears.

- Inform **credit policy adjustments** by identifying vulnerable customer groups and refining lending criteria.
- Strengthen **forecasting and capital adequacy planning** through early detection of repayment stress.
- Support **proactive risk management** by combining portfolio-level trends with segment-specific insights.

In [7]:

```
# Arrears rates by income band show higher risk among  
lower-income customers.  
segment_arrears =  
full_data.groupby(["DATE", "IncomeBand"])  
["ARREARS"].apply(lambda x: (x>0).mean()).unstack()  
  
segment_arrears.plot(marker="o", figsize=(10,6))  
plt.title("Arrears Rate by Income Band")  
plt.ylabel("Rate")  
plt.close()
```

Collections Discipline vs Customer Satisfaction

Focus:

- Compare NPS scores by **Arrears Status**
 - Compare NPS scores by **Default Status**
 - Compare NPS scores by **Account Status**
(Active, Paid Off, etc.)
-

Findings:

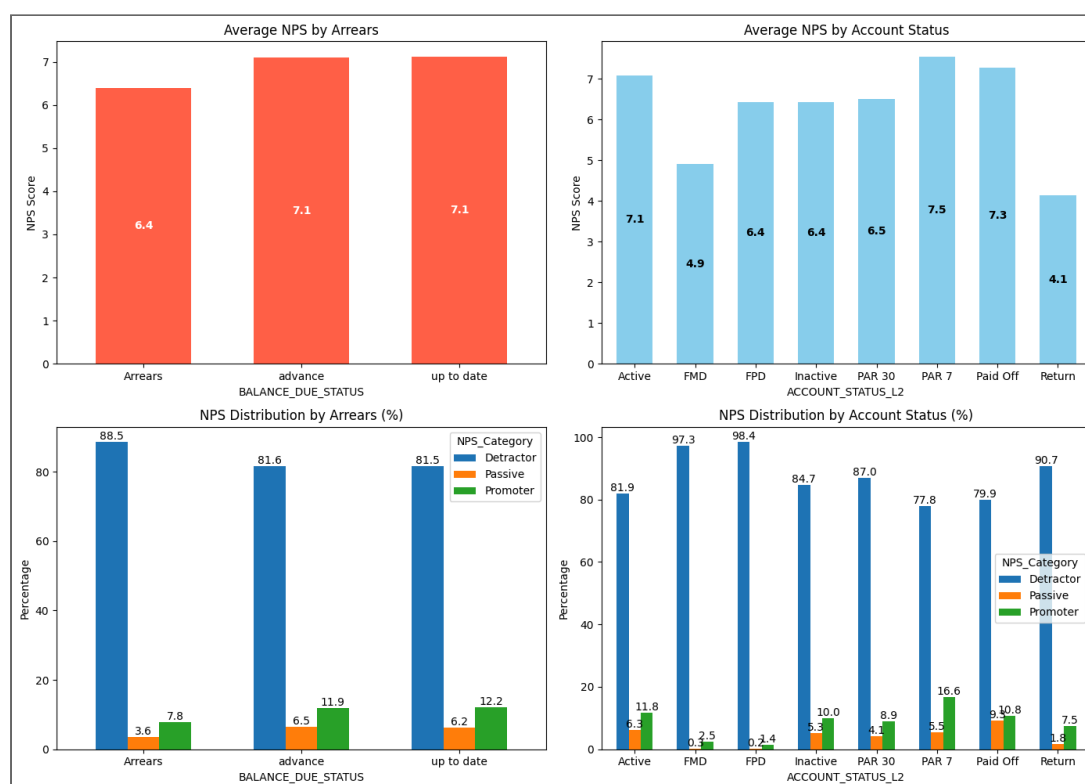
- Customers in **Arrears** report lower NPS, reflecting dissatisfaction with collections enforcement.
- **Defaulted accounts** show the lowest NPS, dominated by detractors due to strict enforcement actions.
- **Paid-off accounts** have the highest NPS, indicating positive closure and restored satisfaction.
- **Active accounts** show moderate NPS, with mixed experiences depending on repayment consistency.

- Overall trend: **the stricter the collections status, the lower the satisfaction**, confirming a direct tension between repayment enforcement and customer trust.

Collections Discipline vs Customer Satisfaction (Cont.)

Visual Evidence:

The chart below illustrates how stricter collections statuses (arrears, defaults, enforcement actions) correspond with lower NPS scores, confirming the trade-off between repayment discipline and customer trust.



The Tension:

- **Collections effectiveness** (arrears flags, phone locks, default enforcement) ensures repayment discipline.

- But these same actions **damage customer experience**, producing more detractors and lowering NPS.
 - The charts clearly show: the stricter the collections status, the lower the satisfaction.
-

This continuation slide reinforces the earlier findings with visual evidence, making the tension between collections effectiveness and customer satisfaction clear and actionable.

In [8]:

```
# Rename NPS column for clarity
full_data.rename(columns={
    'Using a scale from 0 (not likely) to 10 (very
    likely), how likely are you to recommend MoPhones to
    friends or family?': 'NPS Score'
}, inplace=True)

# Categorize NPS scores into Promoter, Passive,
    Detractor
def categorize_nps(score):
    if score >= 9: return "Promoter"
    elif score >= 7: return "Passive"
    else: return "Detractor"

full_data["NPS_Category"] = full_data["NPS
Score"].apply(categorize_nps)

# Average NPS by arrears status
nps_by_arrears =
full_data.groupby("BALANCE_DUE_STATUS")["NPS
Score"].mean()
```

Data Quality – Limitations

Key challenges encountered in MoPhones' datasets:

- **Missing IDs** – Several sheets lack consistent unique identifiers (LoanID/CustomerID), making merges error-prone and increasing risk of duplicates.
- **Too Many Variables** – Redundant fields and poorly defined variables add noise, slow analysis, and obscure key indicators.
- **Unstructured, Fragmented Data** – Credit snapshots, demographics, and NPS survey data are spread across multiple sheets with inconsistent formatting, making integration time-consuming.
- **Ambiguity in Account Status Codes** – Overlapping categories (Active, Inactive, PAR 7, PAR 30, Return) are not mutually exclusive, reducing clarity in portfolio monitoring.

Impact: *These limitations slow down monitoring, increase risk of errors, and weaken the reliability*

of insights drawn from the portfolio data.

Recommendations for Improvement

Proposed actions to strengthen MoPhones' credit data structure:

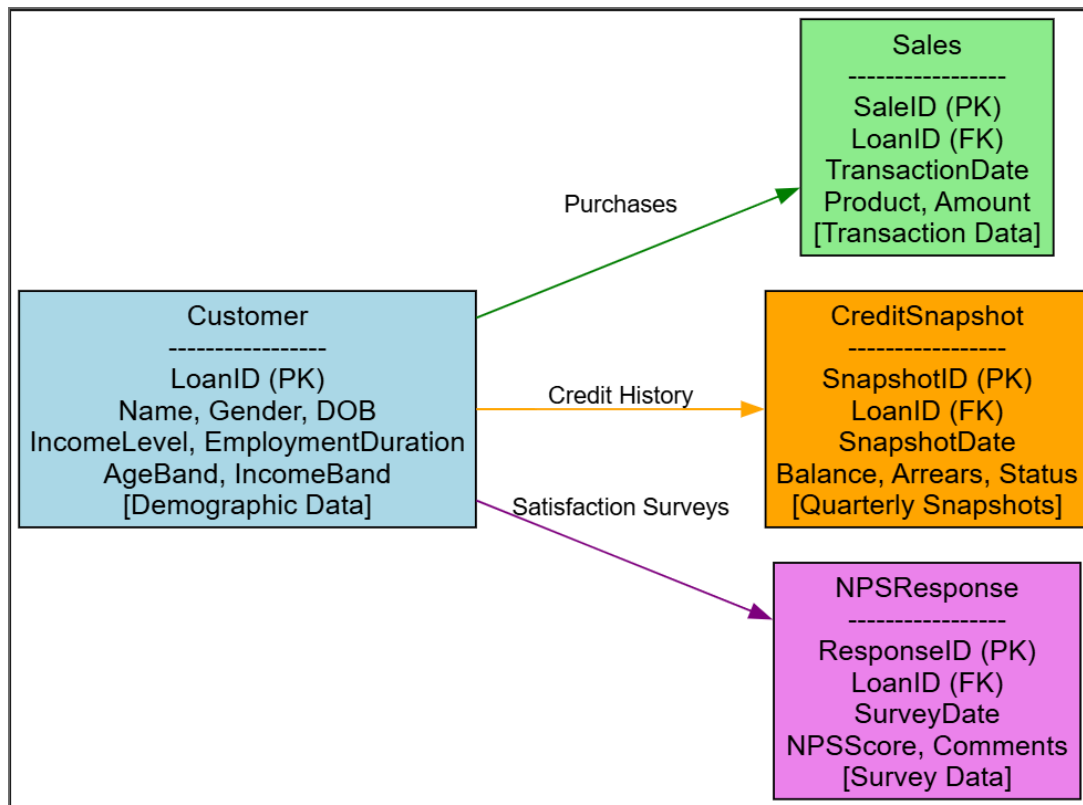
- **Introduce Consistent Unique IDs** – Standardize LoanID and CustomerID across all datasets to enable seamless merging and reduce duplication.
- **Streamline Variables** – Audit and remove unnecessary fields, focusing on core metrics such as loan performance, repayment history, arrears status, and NPS etc,.
- **Consolidate Data Capture** – Replace fragmented sheets with a single relational database or structured workbook using standardized column names and formats.
- **Standardize Account Status Definitions** – Define clear, mutually exclusive categories (Active, Paid Off, Arrears, Default) and nest PAR flags under arrears for consistency.

Objective: *Build a cleaner, more reliable credit monitoring system that reduces analyst time*

spent on data preparation and supports better decision-making.

MoPhones Data Model for Repeatable Reporting

- **Customer** is the central entity, keyed by **LoanID**, holding demographic and segmentation fields.
- **Sales** records transactional data linked to each customer via LoanID.
- **CreditSnapshot** provides quarterly portfolio health metrics (balance, arrears, repayment status).
- **NPSResponse** captures customer satisfaction and feedback, tied back to LoanID.
- **LoanID** serves as the unifying key across all datasets, enabling consistent joins.
- This structure supports **repeatable reporting** by aligning demographics, transactions, credit outcomes, and customer experience.



In [9]:

```
# %pip install graphviz
from graphviz import Digraph
from IPython.display import SVG

# Initialize diagram with SVG output
erd = Digraph('MoPhones_ERD', format='svg')
erd.attr(rankdir='LR', size='8,5', fontname="Arial")

# Define entities with professional font, larger size,
and colors
erd.node('Customer',
        "Customer\n"
        "-----\n"
        "LoanID (PK)\n"
        "Name, Gender, DOB\n"
        "IncomeLevel, EmploymentDuration\n"
        "AgeBand, IncomeBand\n"
        "[Demographic Data]",
        shape='box', style='filled',
fillcolor='lightblue',
        fontname="Arial", fontsize="14")
```

Out[9]:

'MoPhones_ERD.svg'